BUKU SAKU SIDANG PROPOSAL

Rancang Bangun Sistem Speech-to-Text Berbasis Web Menggunakan Model OpenAI Whisper

Muhammad Iffan Ramadhan (2117020036) — Sistem Informasi

---

1. Elevator Pitch (30 Detik)

Inti penelitian: *"Saya membangun sistem transkripsi audio mandiri (self-hosted) yang murah dan aman untuk institusi, menggunakan model AI Whisper. Saya mengukur seberapa akurat dan cepat model ini pada percakapan bahasa Indonesia (data siniar), lalu menguji seberapa mudah antarmuka koreksinya digunakan manusia. Singkatnya: membangun sistem, mengukur performanya, dan menguji kenyamanan penggunanya."*

Logika 1 kalimat: Saya membangun alat transkripsi mandiri, lalu membuktikan alat itu akurat, cepat, dan mudah dipakai.

2. Akar Masalah (3 Lapis)

Lapis	Masalah	Siapa yang Merasakan
1. Praktis	Transkripsi manual sangat lambat (5 jam per 1 jam audio)	Peneliti, jurnalis, institusi
2. Infrastruktur	Layanan cloud ASR mahal & berisiko kebocoran data sensitif	Institusi lokal
3. Teknis	Whisper belum tentu akurat pada percakapan alami bahasa Indonesia (gaul, noise, spontan) — perlu dibuktikan empiris	Peneliti ASR/akademik

Frasa kunci: *"Masalahnya bukan 'belum ada AI', tapi 'AI-nya belum terbukti bagus untuk konteks kita dan belum ada cara memakainya secara mandiri dan terjangkau'."*

3. Tujuan Sebenarnya

- **BUKAN** tujuan: "Membuat aplikasi web keren."
- **ADALAH** tujuan:
    1. **Rancang bangun** sistem STT web terdistribusi (Next.js + Python Worker) — *kontribusi rekayasa.*
    2. **Evaluasi empiris** performa 7 varian Whisper pada data siniar bahasa Indonesia (WER, CER, RTF, VRAM) — *kontribusi ilmiah/data.*
    3. **Evaluasi usability** antarmuka koreksi dengan SUS — *kontribusi HCI.*

Kalimat siap pakai: *"Tujuan saya bukan sekadar membangun aplikasi. Saya menghasilkan (1) sebuah arsitektur referensi, (2) data benchmark baru untuk bahasa Indonesia pada data spontan, dan (3) bukti usability — tiga hal yang bisa dipakai peneliti lain."*

4. Cheat Sheet MATT (1 Menit)

Aspek	Jawaban
Masalah?	Transkripsi manual lambat; cloud mahal & tidak aman; Whisper belum terbukti di bahasa Indonesia spontan.
Apa yang dibangun?	Sistem STT web self-hosted, arsitektur terdistribusi (Next.js + Python Worker + Redis).
Tujuan?	(a) Rancang bangun, (b) benchmark 7 varian Whisper (WER/CER/RTF/VRAM), (c) uji usability (SUS).
Teknis singkat?	WhisperX pipeline + Faster-Whisper di RTX A5000, data 11.540 segmen siniar, 30 responden SUS.

5. Analogi Cepat (Untuk Penguji Non-Teknis)

Istilah	Analogi
ASR / Whisper	"Mesin yang mengubah suara jadi teks"
WER/CER	"Seberapa sering salah dengar — dihitung per kata & per huruf"
RTF	"Kecepatan: kalau RTF 0,5, audio 1 jam selesai dalam 30 menit"
VRAM	"Seberapa besar memori kartu grafis yang dibutuhkan"
Arsitektur terdistribusi	"Seperti restoran: kasir (web) terpisah dari dapur (GPU worker) — pesanan antre, dapur masak, kasir tetap layani pelanggan lain"
Worker	"Koki yang masak di dapur"
Task queue (Redis)	"Papan antrean pesanan di dapur"
Self-hosted	"Masak di dapur sendiri, bukan pesan dari luar"
UCD	"Desain yang dimulai dari tanya dulu ke pengguna, baru bikin"
SUS	"Kuesioner baku untuk nilai kegunaan (skor 0–100)"

6. Strategi Jika Presentasi Gagal / Penguji Bingung

6.1. Teknik "Turun Satu Level"

Jika penjelasan mulai berbelit, berhenti lalu ulangi dengan analogi: *"Baik, saya sederhanakan. Transkripsi itu seperti membuat subtitle. Manual = ngetik sambil dengerin, sangat lama. AI Whisper = bikin draft otomatis, cepat tapi kadang salah, terutama untuk bahasa gaul Indonesia. Penelitian saya: bikin tempat kerja (sistem) supaya AI bisa jalan sendiri di komputer institusi, lalu saya uji seberapa bagus dan seberapa cepat hasilnya untuk podcast bahasa Indonesia, dan seberapa nyaman orang*

mengoreksi hasilnya."

6.2. Teknik "Tunjukkan Keterkaitan"

Jika penguji bertanya "Kok pakai 3 hal sekaligus?": *"Ketiganya bukan topik terpisah. Sistem (arsitektur) adalah wadahnya; benchmark adalah bukti mesinnya bagus; usability adalah bukti manusia bisa memakainya. Tanpa salah satu, klaim 'sistem transkripsi yang baik' tidak lengkap."*

7. Daftar Pertanyaan & Jawaban

7.1. Pertanyaan Dasar (Wajib Hafal)

Q: Apa kontribusi/novelty penelitian Anda?

A: K

ontribusinya bukan algoritma baru, tapi: (1) data benchmark Whisper pada *spontaneous Indonesian speech* yang jarang diuji, (2) arsitektur referensi sistem STT self-hosted terdistribusi untuk institusi lokal, (3) integrasi evaluasi usability (SUS) pada sistem nyata. Tiga hal ini jarang digabung dalam satu penelitian.

Q: Apa bedanya dengan penelitian yang sudah ada?

A: P

enelitian-penelitian tersebut (1) mengukur Whisper pada data formal (bukan siniar spontan), (2) hanya mengukur akurasi tanpa membangun sistem, atau (3) membangun sistem cloud (bukan self-hosted). Penelitian saya menggabungkan ketiganya pada konteks bahasa Indonesia dan institusi lokal.

Q: Kenapa memakai data siniar (podcast)?

A: K

arena siniar adalah representasi percakapan alami — ada bahasa gaul, noise, interupsi, kecepatan bicara bervariasi. Ini justru tantangan yang belum banyak diuji untuk Whisper bahasa Indonesia, dan relevan dengan kebutuhan nyata (banyak siniar/wawancara butuh transkripsi).

Q: Kenapa 7 varian model? Apa gunanya?

A: K

arena pengguna institusi perlu tahu trade-off: model kecil = cepat & hemat VRAM tapi kurang akurat; model besar = akurat tapi butuh GPU mahal. Hasil benchmark menjawab: "varian mana yang paling cost-effective untuk skala institusi lokal?" Ini keputusan praktis yang berguna.

Q: Apa metrik utamanya?

A: E

mpat: WER & CER (akurasi), RTF (kecepatan), VRAM (sumber daya). Kombinasi ini menangkap kualitas DAN kelayakan operasional.

7.2. Pertanyaan Menengah (Perlu Paham Konsep)

Q: Apa itu RTF? Bagaimana membacanya?

A: R

TF = waktu proses ÷ durasi audio. RTF < 1 berarti lebih cepat dari real-time (audio 1 jam diproses kurang dari 1 jam). Ini standar industri ASR.

Q: Apa beda Whisper asli vs WhisperX vs Faster-Whisper?

A: T

iga lapis: **Whisper** = model AI asli OpenAI. **Faster-Whisper** = implementasi ulang yang lebih cepat (C++/CTranslate2). **WhisperX** = pipeline yang menggabungkan VAD (potong audio di hening), Faster-Whisper (transkripsi batch), forced alignment (timestamp akurat), dan diarization (pisah pembicara). Sistem saya memakai WhisperX sebagai pipeline-nya.

Q: Kenapa tidak pakai cloud (Google/Azure API) saja?

A: D

ua alasan: biaya jangka panjang untuk volume besar, dan keamanan — data audio institusi sensitif tidak nyaman dikirim ke server pihak ketiga. Self-hosted memberi kontrol penuh. Inilah justifikasi arsitektur terdistribusi.

Q: Kenapa arsitekturnya dipisah (web vs worker)?

A: K

arena inferensi GPU berat dan lama. Jika digabung, web server bisa membeku. Dengan pemisahan: pengguna tetap bisa melihat antarmuka, mengunggah, dan mengoreksi sementara worker memproses antrean di belakang layar. Ini juga memungkinkan skala horizontal (tambah worker).

Q: Apa itu Human-in-the-Loop di penelitian Anda?

A: A

SR tidak 100% akurat, terutama bahasa gaul. Jadi hasil draf otomatis harus dikoreksi manusia. Sistem saya menyediakan antarmuka koreksi — inilah titik HITL: manusia validator akhir, dan antarmukanya dirancang dengan UCD agar koreksi efisien.

Q: Kenapa pakai SUS? Bisa juga kan metode lain?

A: S

US dipilih karena: (1) terstandarisasi & valid, (2) murah dan cepat (10 item), (3) banyak dipakai sehingga hasilnya bisa dibandingkan lintas studi, (4) memberi skor kuantitatif 0–100 yang mudah diinterpretasikan.

Q: Responden 30 orang — cukup?

A: U

ntuk penelitian usability, 30 responden sudah umum dan memadai untuk mendeteksi masalah utama antarmuka serta memberikan variasi latar belakang pengguna (mahasiswa, akademisi, kreator konten).

7.3. Pertanyaan Sulit / Jebakan (Uji Pemahaman & Kejujuran)

Q: Menurut Anda, bagian mana yang paling lemah dari penelitian ini?

A: K

elemahan utamanya adalah generalisasi: hasil benchmark hanya berlaku untuk data siniar bahasa Indonesia dan GPU RTX A5000. Namun justru karena itu saya membatasi klaim secara jelas di batasan masalah. Ke depan, bisa diperluas ke domain lain (mis. wawancara medis/hukum) dan GPU lain.
(Jangan jawab "tidak ada kelemahan" — penguji menghargai kesadaran akan limitasi.)

Q: Ini kan cuma bikin aplikasi, mana penelitiannya?

A: S

aya menerima tantangan ini. Bedanya dengan sekadar bikin aplikasi: (1) ada pertanyaan empiris yang diuji — seberapa baik Whisper pada data spontan bahasa Indonesia, yang belum banyak datanya; (2) ada pengukuran sistematis (WER/CER/RTF/VRAM) dengan ground truth; (3) ada evaluasi usability terstandarisasi; (4) semua dipublikasikan sebagai arsitektur referensi. Aplikasinya hanyalah kendaraan untuk menjawab pertanyaan-pertanyaan tersebut.
(Ini pertanyaan paling penting — persiapkan mental.)

Q: Kenapa tidak sekalian fine-tune Whisper supaya akurat?

A: F

ine-tuning memang bisa, tapi di luar scope proposal ini. Fokus saya adalah evaluasi model yang sudah ada + membangun sistem yang bisa memakainya. Fine-tuning layak menjadi penelitian lanjutan (future work) — dan justru data benchmark saya bisa jadi dasar untuk itu.
Alternatif: "Fine-tune butuh data berlabel besar dan biaya komputasi tambahan. Sebelum fine-tune, kita perlu tahu dulu baseline-nya — itulah yang saya ukur."

Q: Bagaimana Anda menjamin hasil benchmark valid/reliable?

A: S

aya: (1) memakai ground truth TSV yang sudah divalidasi, (2) memastikan kesesuaian durasi audio vs teks, (3) menjalankan pengujian berulang di lingkungan terkontrol (Runpod RTX A5000), (4) memakai metrik standar industri (WER/CER/RTF).

Q: Apa implikasi praktis dan teoritis dari hasil Anda nanti?

A: T

eoritis: menambah peta performa Whisper pada bahasa Indonesia, khususnya speech spontan. Praktis: institusi bisa memilih varian model yang cost-effective berdasarkan kebutuhan — misal butuh cepat vs butuh akurat — dan bisa menerapkan arsitektur serupa.

Q: Apa yang terjadi jika semua model hasilnya buruk?

A: H

asil "buruk" pun tetap bernilai ilmiah — ia menunjukkan batas kemampuan Whisper pada domain ini dan memperkuat argumen perlunya fine-tuning atau post-processing. Penelitian tidak wajib hasilnya bagus; yang penting terukur dan jujur.

Q: Kenapa memilih Next.js, Python, Redis, PostgreSQL — bukan yang lain?

A: P

ilihan ini berdasarkan kebutuhan: Next.js + TypeScript untuk antarmuka yang cepat dan type-safe; Python karena ekosistem ASR (WhisperX/Faster-Whisper) dominan di Python; Redis sebagai task queue ringan dan real-time status; PostgreSQL karena data transkripsi relasional. Ini pemilihan yang lazim dan matang di industri.

Q: Berapa lama Anda memperkirakan penelitian ini selesai?

A: D

elapan minggu sesuai jadwal: 2 minggu studi literatur & pengumpulan data, 2 minggu perancangan & implementasi, 1 minggu pengujian fungsional, 1 minggu benchmarking, 1 minggu usability, 1 minggu laporan.

8. Peta Konsep 3 Pilar (Untuk Ditunjukkan ke Penguji)

Pilar 1: Mesin Transkripsi	**Pilar 2: Fondasi Sistem**	**Pilar 3: Antarmuka Koreksi**
Model Whisper via WhisperX	Arsitektur terdistribusi Next.js + Python Worker + Redis	UCD + evaluasi SUS
Menjawab RQ2 (performa)	Menjawab RQ1 (rancang bangun)	Menjawab RQ3 (usability)

Selamat Sidang! — Hafalkan Elevator Pitch, Akar Masalah 3 Lapis, dan Jawaban "Ini Bukan Cuma Bikin Aplikasi".